

递归特权架构：可扩展隔离的统一模型

作者姓名

所属机构

摘要—现代处理器特权架构面临一个根本性限制：特权等级由硬件固定，无法按需扩展。x86 提供四个固定的特权环，ARM TrustZone 提供两个隔离世界，RISC-V 定义三种特权模式。每种新需求——虚拟化、可信执行、内存保护——都需要独立的硬件扩展，导致架构碎片化。本文提出递归特权架构 (Recursive Privilege Architecture, RPA)，将特权配置从硬件固定转变为内存可配置。RPA 引入两个对称原语 (descend、ascend)，支持由系统资源而非硬件约束决定的理论无限特权层级。通过页表叠加，RPA 原生支持嵌套虚拟化；通过 INHERIT 模式，系统调用开销接近函数调用延迟；通过安全域机制，RPA 实现了机密计算中管理权与访问权的分离。包含 71 个测试用例的 Python 可执行规范验证了所有原语的功能正确性。与 x86 Ring、ARM TrustZone 和 RISC-V PMP 相比，RPA 为虚拟化、机密计算和沙箱场景提供了统一抽象。

Index Terms—特权架构，虚拟化，机密计算，内存保护，处理器设计

1. 引言

现代处理器架构通过固定的硬件机制实现特权分离。x86 定义四个特权环 (Ring 0–3)，ARM TrustZone 提供两个隔离世界 (安全世界/正常世界)，RISC-V 规定三种特权模式 (机器态/监管态/用户态)。虽然这些机制在其原始设计目标上有效，但它们都有一个根本性限制：特权等级的数量和语义无法在没有新硬件的情况下扩展。

这一限制导致了架构碎片化。当虚拟化需要额外的特权等级时，Intel 引入了 VT-x (VMX root/non-root 模式)，AMD 开发了 AMD-V，ARM 设计了虚拟化扩展。当可信执行环境 (TEE) 成为必要时，Intel 创建了 SGX Enclave 和 TDX 信任域，AMD 开发了 SEV/SNP，ARM 正在构建 CCA Realm。当需要

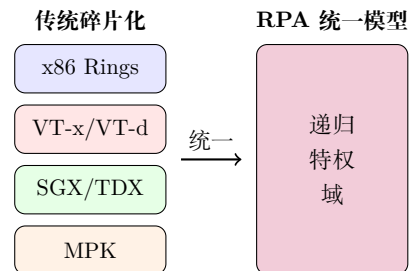


图 1. 特权机制碎片化与 RPA 统一模型。传统架构为每种用例需要独立扩展；RPA 提供单一递归模型。

更细粒度的内存保护时，Intel 添加了内存保护密钥 (MPK)。每种需求都催生了独立的扩展，如图 1 所示。

本文提出递归特权架构 (RPA)，通过三个关键贡献解决这一限制：

贡献 1：递归特权模型。 RPA 引入相对视角概念：每个特权域将自己视为 Domain 0，其子域为 Domain 1, 2, 3, ..., 其父域为 Domain -1。这种递归定义支持理论无限的嵌套深度，仅受系统资源（域控制块的内存）限制，而非硬件设计。

贡献 2：统一特权原语。 RPA 定义两个对称原语：descend 进入或恢复子域，ascend 返回父域。这两个原语替代了传统架构的异构机制：syscall/sysret、VM entry/exit、SMC（安全监控调用）和异常处理都成为 descend/ascend 的实例。

贡献 3：ISA 无关设计。 RPA 可以叠加在现有指令集 (ARM、RISC-V、x86) 之上，而不需要完全重新设计 ISA。只需要两条新指令；所有现有代码无需修改即可运行。控制块位于普通内存中，通过标准加载/存储操作进行配置。

我们通过包含 71 个测试用例的 Python 可执行规范验证 RPA，覆盖：域操作 (descend、ascend)、单

层和多层页表翻译、IPA 边界检查、安全组机制和中断处理。该工件可在 [1] 获取。

本文其余部分组织如下。第 2 节回顾相关工作。第 3 节介绍 RPA 架构。第 4 节演示部署模式。第 5 节将 RPA 与现有架构进行比较。第 6 节讨论评估方法。第 7 节讨论局限性和未来工作。第 8 节总结全文。

2. 背景与动机

2.1. 传统特权架构

x86 保护环。x86 架构通过代码段选择器中的当前特权级别 (CPL) 字段实现四个特权环。Ring 0 (内核态) 拥有完全硬件访问权限, Ring 3 (用户态) 受限。环间转换通过 `syscall/sysret` 指令进行, 典型开销为 100–500 周期 [2]。虚拟化需要 VT-x 扩展, 引入 VMX root 和 non-root 模式, 实际上增加了两个特权级别。

ARM TrustZone。ARM TrustZone 实现两个隔离世界: 安全世界和正常世界, 通过 SMC (安全监控调用) 指令分隔。世界切换开销为 50–200 周期 [3]。TrustZone 仅支持一个 TEE 实例, 限制了嵌套场景。ARM 即将推出的机密计算架构 (CCA) 引入 Realm 管理扩展, 支持硬件隔离的虚拟机。

RISC-V 特权架构。RISC-V 定义三种特权模式: 机器态 (M)、监管态 (S) 和用户态 (U)。物理内存保护 (PMP) 提供可配置的内存区域, 通常 8–16 个条目。虚拟化需要 H 扩展。RISC-V 的简洁性具有吸引力, 但固定的特权层级仍然存在。

2.2. 硬件隔离扩展

Intel 内存保护密钥 (MPK)。MPK 通过 PKRU 寄存器控制的 16 个保护域提供用户态内存隔离。通过 `WRPKRU` 指令, 域切换开销仅 1–10 周期 [4]。然而, MPK 不提供对内核访问的保护, 也不支持加密。

Intel SGX 和 TDX。SGX 创建 Enclave——由内存控制引擎 (MCE) 加密的硬件隔离内存区域。Enclave 大小受 EPC (Enclave 页面缓存) 限制, 通

常为 128 MB。TDX 将其扩展到 VM 级信任域, 支持多密钥内存加密 [5]。两者都不支持嵌套。

CHERI 能力架构。CHERI 在 ISA 中引入能力——携带权限的不可伪造令牌——提供细粒度内存安全 [6]。CHERI 解决内存安全问题, 而 RPA 专注于特权层级; 两者正交且可能互补。

2.3. 嵌套虚拟化挑战

嵌套虚拟化——在虚拟机内运行虚拟机监控器——需要软件模拟硬件虚拟化特性。这种模拟产生显著开销:

- 影子页表需要维护客户页表的软件副本
- 嵌套 EPT (扩展页表) 增加翻译层级
- 每个嵌套层级的 VM exit 增加延迟

Ben-Yehuda 等人 [7] 测量了使用影子页表的嵌套虚拟化开销为 10–30%。该开销随嵌套深度增长。

2.4. 动机: 可扩展特权的必要性

硬件扩展的激增揭示了一种设计模式: 每个新的隔离需求都迫使新硬件。这种模式表明需要一种统一抽象, 它:

- 1) 支持动态特权深度而无需重新设计硬件
- 2) 在各种用例 (虚拟化、TEE、沙箱) 中提供一致的原语
- 3) 在不需要特权分离时实现高效转换
- 4) 允许在现有 ISA 上渐进部署

RPA 通过递归特权域解决这些需求。

3. RPA 架构

本节介绍递归特权架构: 递归特权模型 (第 3.1 节)、核心原语 (第 3.2 节)、域控制块 (第 3.3 节)、页表叠加 (第 3.4 节) 和安全域 (第 3.5 节)。

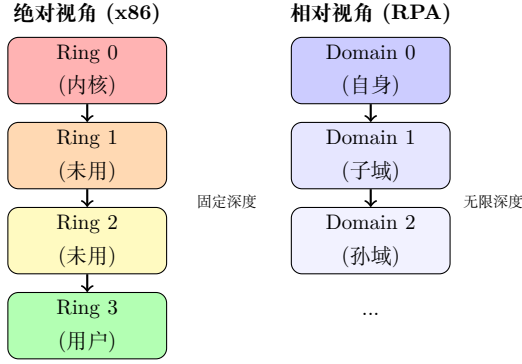


图 2. 绝对特权级别 (x86) 与相对视角 (RPA) 对比。x86 环由硬件固定；RPA 域递归定义。

3.1. 递归特权模型

相对视角。 RPA 的根本创新是相对视角：每个域将自己视为 Domain 0，其子域为 Domain 1, 2, 3, ...，其父域为 Domain -1。这与传统架构中特权级别为绝对值 (Ring 0, Ring 1, ...) 形成对比。

图 2 展示了这一区别。传统 x86 有四个固定环；RPA 支持无限嵌套，每个域具有相同的视图结构。

域层级。 RPA 将域组织为树结构：

- **Domain 0:** 信任根，由外部认证建立
- **Domain $N + 1$:** Domain N 的子域，由父域的 descend 操作创建
- 更高的域编号表示更低的特权 (特权沿树下降)

层级深度受域控制块内存限制，而非硬件限制。一个为控制块分配 128 KB 的系统可支持超过 1000 个嵌套域。

3.2. 核心原语

RPA 定义两个对称原语，形成优雅的特权转换调用/返回模型：

descend(block_addr)。 进入子域，从指定 DomainBlock 地址读取配置：

- 1) 验证控制块对齐 (32 字节边界) 和大小
- 2) 检查 child_block 判断首次进入还是恢复
- 3) 首次进入：保存当前上下文，加载新 DomainBlock，转移到入口点

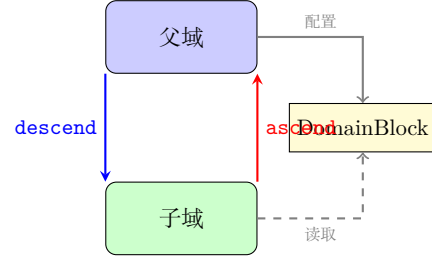


图 3. RPA 的两个对称原语。descend 进入或恢复子域；ascend 返回父域。父域配置子域的 DomainBlock；子域无法修改自己的控制块。

4) 恢复：恢复子域保存的上下文并继续执行

ascend(service_type)。 返回父域以请求服务或报告事件：

- 1) 将当前上下文保存到子域的 DomainBlock
- 2) 恢复父域的 DomainBlock 为活动状态
- 3) 跳转到父域的 trap_vector
- 4) 在指定寄存器 (如 R0) 中传递 service_type

对称性是有意设计的：descend 和 ascend 是逆操作。父域调用 descend 时挂起自己；子域调用 ascend 时挂起自己。两个操作都保存上下文、切换域并恢复对方。

这两个原语替代了传统的特权转换机制：

- **系统调用：** 用户域调用 ascend 请求内核服务
- **VM entry/exit：** 客户 VM 使用 ascend；虚拟机监控器使用 descend 返回
- **TEE 切换：** 正常世界调用 descend 进入安全世界
- **异常处理：** 异常触发隐式 ascend

3.3. DomainBlock 结构

DomainBlock 是驻留在内存中的 128 字节控制结构，对齐到 32 字节边界。表 1 展示了关键字段。

安全原则： 父域配置子域的 DomainBlock；子域无法修改自己的控制块。这确保：

- 父域控制子域的 IPA 边界 (内存访问限制)
- 父域控制子域的入口点 (在 ctx_save_area 中)
- 子域可以设置自己的异常处理器 (exception_vector) 和页表

表 1. DOMAINBLOCK 关键字段

偏移	字段	设置者	用途
0x00	ctrlblock_size	父域	控制块大小
0x04	exception_vector	子域	异常处理入口
0x0C	interrupt_ctrl	系统	中断控制器句柄
0x10	ipa_regions	父域	IPA 边界表
0x14	domain_id	系统	唯一域标识符
0x18	pagetable	子域	页表基地址
0x1C	child_block	父域	子域控制块地址
0x20	security_domain	系统	安全域句柄
0x28	ctx_save_area	硬件	ISA 相关上下文保存区 (96 字节)

- 硬件在转换期间自动保存/恢复上下文

ISA 无关性: DomainBlock 的大部分字段与 ISA 无关。ctx_save_area (96 字节) 保存 ISA 相关的寄存器状态：在 ARM 上存储栈指针、链接寄存器和程序状态；在 x86 上存储栈指针、指令指针和标志寄存器；在 RISC-V 上存储 sp、ra 和状态寄存器。具体的寄存器到偏移映射由各 ISA 定义，但整体结构和语义保持一致。

3.4. 页表叠加

RPA 支持页表叠加：每个域可以有自已的页表，将域的”物理”地址翻译到父域的地址空间。

术语：

- **VA:** 虚拟地址——当前域的虚拟地址
- **IPA:** 中间物理地址——一级翻译的结果
- **PA:** 物理地址——最终内存地址

翻译过程: 对于 Domain $N + 1$ 访问地址 VA:

- 1) 通过 PT_{N+1} 翻译 VA 得到 IPA_N
- 2) 检查 IPA_N 是否在 Domain N 定义的 ipa_regions 内
- 3) 如果有效，继续通过 PT_N 翻译
- 4) 重复直到到达物理内存

IPA 边界强制: 父域定义 ipa_regions 来约束子域的可访问地址范围。如果子域创建的页表条目映射到 ipa_regions 之外，硬件触发翻译错误。此机制确保：

- 子域无法访问未明确授予的内存

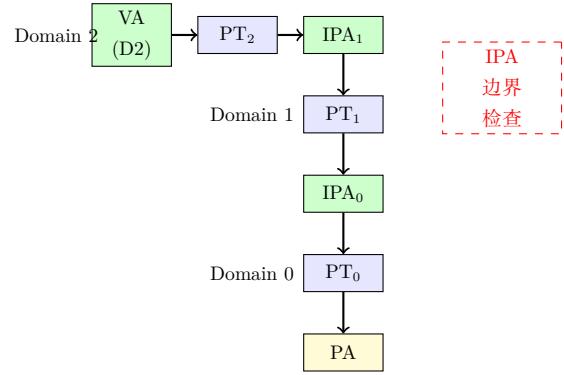


图 4. 页表叠加。每个域的页表翻译到父域的地址空间。IPA 边界检查防止子域访问未授权区域。

- 父域可以通过修改 ipa_regions 撤销访问
- 内存隔离由硬件强制

INHERIT 模式: 当 pagetable = 0 时，子域共享父域的地址空间。此模式具有近乎零的转换开销：

- 无页表切换
- 无 TLB 刷新
- 仅保存/恢复寄存器上下文

INHERIT 模式实现高效系统调用和异常处理，在第 4.2 节讨论。

3.5. 安全域

RPA 引入安全域用于机密计算场景，区分管理权和访问权。

管理权与访问权：

- **管理权:** 父域可以配置子域的 ipa_regions、页表、入口点
- **访问权:** 父域无法读取子域的私有内存内容

这种分离实现了一种新颖的机密计算模型：

- 1) 父域(如云提供商)创建带有 security_domain 标志的子域
- 2) 内存控制器使用硬件派生密钥加密子域内存
- 3) 父域可以管理资源 (CPU、内存分配) 但无法读取数据
- 4) 硬件保证只有安全域持有者可以解密

与 Intel TDX 或 AMD SEV 相比，RPA 安全域：

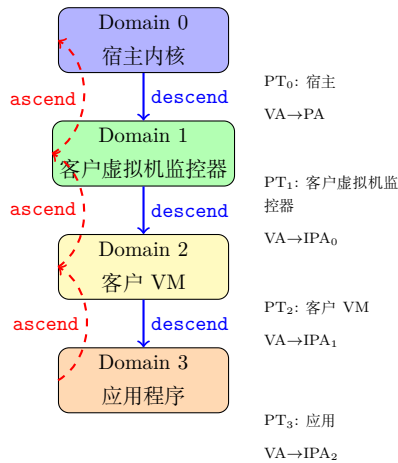


图 5. 使用 RPA 的嵌套虚拟化。每级有自己的页表；VM exit 成为 **ascend** 操作，具有多层翻译的原生硬件支持。

- 支持嵌套（安全域内的安全域）
- 使用与普通域相同的原语
- 不需要单独的 ISA 扩展

4. 部署模式

本节演示 RPA 统一模型支持的四种部署模式：嵌套虚拟化（第 4.1 节）、系统调用优化（第 4.2 节）、机密计算（第 4.3 节）和异常处理（第 4.4 节）。

4.1. 嵌套虚拟化

嵌套虚拟化——在虚拟机内运行虚拟机监控器——需要模拟硬件虚拟化特性。传统方法使用影子页表或嵌套 EPT，产生 10–30% 开销 [7]。

RPA 通过域层级原生支持嵌套虚拟化：

相比传统嵌套虚拟化的优势：

- 1) **原生多层翻译**：硬件遍历所有页表，消除影子页表开销
- 2) **统一 VM exit 路径**：客户虚拟机监控器的 VM exit 成为 **ascend** 到 Domain 1，而非 Domain 0
- 3) **可配置嵌套深度**：仅受控制块内存限制，而非硬件限制

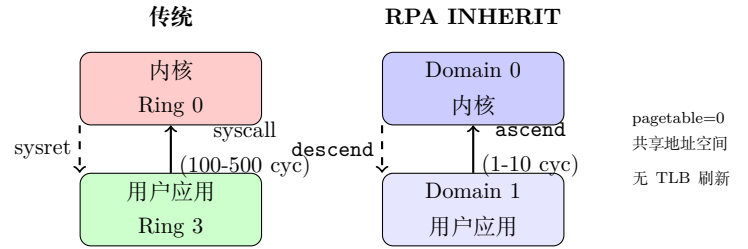


图 6. 系统调用优化。INHERIT 模式共享地址空间，消除页表切换和 TLB 刷新。

每次 VM exit 触发 **ascend** 到直接父域（客户虚拟机监控器），而非根。这保留了每级的虚拟化语义，无需软件模拟。

4.2. 系统调用优化

传统系统调用涉及：

- 1) 特权级别切换 (Ring 3 → Ring 0)
- 2) 页表切换 (用户 → 内核)
- 3) TLB 刷新 (通常总计 100–500 周期 [2])

RPA 的 INHERIT 模式实现近乎零开销的系统调用：

INHERIT 模式机制：当子域的 `pagetable` 字段为 0 时，子域继承父域的地址空间。**ascend/descend** 循环仅涉及：

- 保存/恢复寄存器 (5–10 周期，仅调用者保存)
- 无页表切换
- 无 TLB 刷新

这使系统调用开销接近函数调用延迟，实现细粒度特权分离而无性能损失。

安全考虑：INHERIT 模式适用于：

- 父域信任子域不会破坏共享数据结构
- 子域代码经过验证或沙箱化
- 内存保护通过 `ipa_regions` 或软件检查强制

对于不可信代码，独立页表以更高的转换开销提供完全隔离。

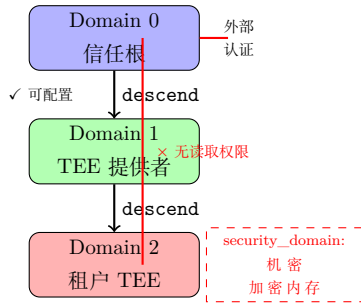


图 7. 安全域的机密计算。Domain 0 可以配置 Domain 2 的 ipa_regions 但无法读取加密内存。

4.3. 机密计算

RPA 的安全域实现了一种新颖的机密计算模型，云提供商管理资源但无法访问租户数据。

管理权与访问权：

- **Domain 1** (TEE 提供者)：配置 Domain 2 的 ipa_regions，分配内存，但无法读取 Domain 2 的数据
- **Domain 2** (租户 TEE)：内存由硬件加密；仅此域可解密
- **Domain 0** (根)：知道 Domain 2 存在但既无管理权也无访问权

此模型与 Intel TDX 和 AMD SEV 不同：

- **嵌套支持**：TEE 内的 TEE 原生支持
- **统一原语**：与非机密域使用相同的 descend/ascend 机制
- **无单独硬件模式**：安全域是 DomainBlock 中的标志，非单独 CPU 模式

4.4. 异常处理

RPA 通过 ascend 提供硬件原生异常处理，实现高效的 try-catch 语义：

工作流程：

- 1) 父域将 exception_vector 设置为 catch_handler
- 2) 父域调用 descend 进入 try 块 (子域)
- 3) 子域执行；如果发生异常，硬件触发 ascend

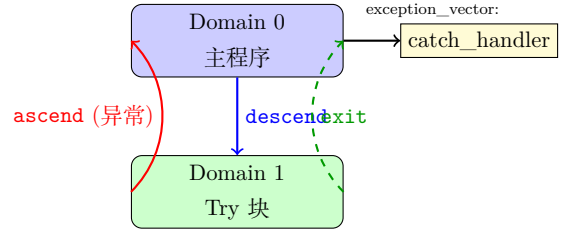


图 8. 使用 RPA 的异常处理。Try 块在子域中执行；异常自动 ascend 到父域的 exception_vector。

- 4) 父域的 catch_handler 接收异常信息
- 5) 处理器要么恢复并调用 descend，要么终止子域

相比软件异常处理的优势：

- **零软件开销**：无异常表查找，无栈展开
- **硬件隔离**：子域内存受保护，防止破坏
- **兼容 INHERIT 模式**：共享地址空间实现零开销正常路径

这种模式将异常处理从语言运行时特性转变为硬件原语，适用于任何语言而无需运行时修改。

5. 与现有架构对比

表 2 从关键维度将 RPA 与现有特权架构进行比较。

5.1. 特权级别可扩展性

x86 Ring 由硬件固定为四个级别。虚拟化需要 VT-x (VMX root/non-root)，实际上增加了两个级别，但通过单独的机制。

ARM TrustZone 提供两个世界：安全世界和正常世界。不支持嵌套；每个安全世界共享单个 TEE 实例。

RISC-V 定义三种特权模式 (M/S/U)。H 扩展增加虚拟化支持，但层级深度仍然固定。

RPA 支持受内存限制的无限嵌套。添加新的隔离级别只需分配新的 DomainBlock，无需重新设计硬件。

表 2. 特权架构对比

特性	x86 Ring	ARM TrustZone	RISC-V PMP	CHERI	RPA
特权级别	固定 (4)	固定 (2)	固定 (3)	无	可配置
转换机制	syscall/sysret	SMC	ecall	能力	descend/ascend
嵌套支持	VT-x 扩展	否	H 扩展	不适用	原生
机密计算	SGX/TDX	TrustZone	定制	定制	安全域
转换开销	100–500 cyc	50–200 cyc	10–100 cyc	1–10 cyc [†]	1–10 cyc [‡]
ISA 修改	否	否	否	显著	最小
内存保护	页级	页级	可配置	对象级	页 + 安全
统一抽象	否	否	否	否	是

[†] 能力加载/存储, 非特权转换 [‡] INHERIT 模式

5.2. 转换机制统一

传统架构对不同隔离场景使用不同机制:

- 系统调用: syscall/sysret (x86)、SVC (ARM)、ecall (RISC-V)
- 虚拟化: VM entry/exit (VT-x、AMD-V)
- TEE: SMC (ARM TrustZone)、ENCLS/ENCLU (Intel SGX)
- 异常处理: 中断/陷阱机制

RPA 将这些统一为两个对称原语: descend 和 ascend。这种统一:

- 1) 简化硬件设计 (三条指令代替多种机制)
- 2) 在各种场景中提供一致的编程模型
- 3) 支持组合 (VM 内的 TEE、TEE 内的异常处理)

5.3. 机密计算支持

Intel SGX 创建具有硬件加密内存的 Enclave, 但大小受限 (EPC) 且不可嵌套。

Intel TDX 扩展到 VM 级信任域, 支持多密钥加密, 但需要单独的硬件模式。

ARM CCA 引入 Realm 作为硬件隔离的 VM, 目前正在开发中。

RPA 安全域提供:

- 嵌套支持 (安全域内的安全域)
- 管理/访问权分离
- 与相同原语集集成 (无需新指令)

5.4. ISA 无关性

CHERI 需要显著的 ISA 更改: 能力寄存器、标记内存、新指令编码。这实现了细粒度内存安全, 但需要新硅片。

RPA 可以叠加在现有 ISA 之上:

- 仅两条新指令 (descend、ascend)
- 控制块位于普通内存 (无新寄存器)
- 页表遍历可复用现有 MMU 逻辑
- 现有软件无需修改即可运行

这支持渐进部署: RPA 可以作为协处理器或微码扩展实现, 然后在未来的设计中迁移到专用硬件。

5.5. 局限性

与现有架构相比, RPA 引入了权衡:

硬件复杂性。多层页表遍历需要专用硬件或软件模拟。这种复杂性今天已存在于嵌套虚拟化中; RPA 将其作为一等特性。

上下文大小。每个域需要 128 字节用于 Domain-Block。深度嵌套 (如 100 级) 消耗 ~12.5 KB, 但这与页表相比可忽略不计。

无内存安全。RPA 提供特权分离, 而非内存安全。CHERI 解决边界违规和释放后使用; RPA 不解决。两种机制正交且可能组合。

迁移路径。现有操作系统和虚拟机监控器需要修改才能使用 RPA 原语。然而, 向后兼容性得以保留: 传统系统调用接口可以使用 ascend 实现。

6. 评估方法

本节介绍我们的评估方法：可执行规范（第 6.1 节）、功能验证（第 6.2 节）、理论性能分析（第 6.3 节）和对比方法（第 6.4 节）。

6.1. 可执行规范

我们将 RPA 实现为约 1,500 行代码的 Python 可执行规范。这种方法遵循架构研究的既定实践：

- CHERI 提供 Sail ISA 规范 [8]
- RISC-V 有正式的 Sail 模型 [9]
- seL4 的 Haskell 原型先于 C 实现 [10]

该规范建模：

- 域操作：descend、ascend
- 内存子系统：页表、IPA 翻译、边界检查
- 安全组：域成员资格和访问控制
- 中断处理：中断控制器、上下文保存/恢复

工件可用性：可执行规范作为开源软件提供 [1]，支持所有测试结果的复现和验证。

6.2. 功能验证

我们开发了 71 个测试用例，覆盖 RPA 核心功能：

表 3. 测试用例类别

类别	测试数量
域操作 (descend、ascend)	18
页表翻译 (单级、多级)	15
IPA 边界检查	12
安全组成员资格和访问	10
中断处理和上下文保存/恢复	8
内存加密模拟	5
异常传播	3

关键测试场景：

- **嵌套虚拟化：**4 级域层级，具有独立页表
- **IPA 违规：**子域尝试访问 ipa_regions 之外的内存

- **安全域隔离：**父域无法读取子域的加密内存
- **异常传播：**深层子域中的故障传播通过所有祖先

全部 71 个测试用例通过，验证了规范的语义正确性。

6.3. 理论性能分析

由于缺乏硬件实现，我们提供转换开销的理论分析：

表 4. 理论转换开销分析

操作	INHERIT	独立页表	传统
寄存器保存/恢复	5–10 cyc	50–100 cyc	100–300 cyc
页表切换	0	50–100 cyc	50–100 cyc
TLB 刷新	0	100–500 cyc	100–500 cyc
总计	5–10 cyc	200–700 cyc	250–900 cyc

传统开销来自 Soares 等人 [2] 和 VM exit 测量 [11]

INHERIT 模式优势：当 pagetable = 0 时，子域共享父域的地址空间。转换开销减少到仅寄存器保存/恢复：

- 调用者保存寄存器：通常 5–10 个寄存器
- 无页表操作
- 无 TLB 失效

这使系统调用开销从数百周期降至数十周期——与函数调用相当。

独立页表模式：当子域有独立页表时，开销与传统系统调用相似：

- 寄存器保存/恢复
- 页表基址寄存器更新
- TLB 刷新或 ASID 切换

优势不在于单次转换开销，而在于嵌套效率：每级增加一个翻译层，无软件模拟开销。

6.4. 对比方法

我们与现有架构的对比（表 2）基于：

- 发布的规范：Intel SDM [4]、ARM 架构参考手册 [3]、RISC-V 特权 ISA [12]

- 学术文献: CHERI 论文 [6]、嵌套虚拟化研究 [7]
- 性能测量: 系统调用开销研究 [2]、VM exit 基准测试 [11]

我们承认硬件实现将提供更准确的性能数据。然而, CHERI 的先例表明, 架构创新可以通过规范、模拟和软件移植在硅片可用之前得到验证 [6]。

7. 讨论

7.1. 硬件实现考虑

控制块存储。 DomainBlock (每个 128 字节) 可以位于:

- 专用 SRAM: 访问最快, 但容量有限
- 标记内存: 类似于 CHERI 的能力标记, 防止伪造
- 带硬件保护的普通内存: 成本最低, 需要额外检查

对于支持 16 个并发域的系统, 2 KB 专用存储足够。深度嵌套 (100+ 级) 需要从主内存动态分配。

页表遍历。多级翻译需要遍历多个页表。两种实现方法:

- 1) **硬件遍历器:** 扩展 MMU 遍历 Domain N 的页表, 然后是 Domain $N - 1$ 的, 依此类推
- 2) **软件辅助:** 硬件遍历一级; 软件处理跨级翻译

硬件遍历将复杂性分摊到多次翻译中; 软件辅助为实验提供灵活性。

TLB 设计。缓存翻译必须包含域 ID 以避免域切换时刷新。域感知的 TLB 条目格式:

[Domain ID | Virtual Page | Physical Page | Permissions]

这类似于 ARM 中的 ASID (地址空间 ID) 和 x86 中的 PCID (进程上下文 ID)。

7.2. 安全考虑

控制块完整性。子域无法修改自己的 DomainBlock。实现选项:

- **内存保护:** DomainBlock 位于仅父域可访问的内存区域
- **标记内存:** 硬件标记识别 DomainBlock 区域
- **哈希验证:** 硬件维护 DomainBlock 的哈希

IPA 边界强制。IPA 边界检查防止子域逃避内存约束。ipa_regions 表由父域配置, 子域只读。硬件必须验证子域创建的每个页表条目。

侧信道。RPA 不解决侧信道攻击 (缓存时序、分支预测)。这与特权分离正交:

- 缓存分区可与 RPA 结合
- 推测执行屏障可在域转换处插入
- 常量时间编程仍是软件的责任

7.3. 与 CHERI 的关系

CHERI 通过能力提供细粒度内存安全; RPA 提供可扩展特权层级。两者互补:

表 5. RPA 与 CHERI 对比

方面	CHERI	RPA
主要目标	内存安全	特权分离
核心机制	能力	域层级
层级深度	无	无限
ISA 修改	显著	最小
硬件成本	标记内存	控制块存储

潜在集成: 组合架构可以:

- 在每个 RPA 域内使用 CHERI 能力实现内存安全
- 使用 RPA descend/ascend 进行特权转换
- 将 CHERI 的密封能力与 RPA 的安全域结合

我们将此集成留作未来工作。

7.4. 局限性与未来工作

无硬件原型。本文提出架构设计, 无硬件实现。遵循 CHERI 的先例 [6], 我们通过规范和功能测试验证。硬件实现 (FPGA 或 ASIC) 留作未来工作。

无操作系统移植。实际验证需要移植操作系统使用 RPA 原语。最小微内核或 seL4 移植将演示:

- 真实工作负载下的系统调用开销
- 嵌套虚拟化性能
- 机密计算集成

无形式化验证。CHERI 和 seL4 使用形式化方法 (Isabelle、Coq) 证明安全属性。用 Sail 或 Lem 对 RPA 原语进行形式化规范, 然后用 Isabelle 证明隔离属性, 将加强安全保证。

中断控制器集成。RPA 的中断控制器接口已草拟但未完全规范。与现有中断架构 (ARM GIC、RISC-V PLIC、x86 APIC) 集成需要额外设计工作。

未来方向:

- 1) Sail 语言的形式化规范
- 2) 安全属性证明 (隔离、特权单调性)
- 3) gem5 模拟以进行周期精确性能估计
- 4) RISC-V 软核上的 FPGA 原型
- 5) 操作系统集成 (seL4 或最小微内核)

8. 结论

我们提出了递归特权架构 (RPA), 一种可扩展隔离的统一模型, 将特权配置从硬件固定转变为内存可配置。RPA 的关键创新包括:

递归特权模型。每个域以相对视角将自己视为 Domain 0。这实现了受内存限制而非硬件限制的无限嵌套深度, 支持从嵌套虚拟化到硬件原生异常处理的场景。

统一原语。两个对称原语——**descend** 和 **ascend**——替代了传统架构的异构机制。系统调用、VM entry/exit、TEE 切换和异常处理都成为这些原语的实例。

ISA 无关性。RPA 可以仅用两条新指令叠加在现有 ISA 之上。控制块位于普通内存, 支持渐进部署和向后兼容。

INHERIT 模式。父域和子域共享地址空间, 将转换开销从数百周期 (传统系统调用) 降至数十周期 (与函数调用相当), 实现细粒度特权分离而无性能损失。

我们通过包含 71 个测试用例的 Python 可执行规范验证了 RPA, 覆盖域操作、页表翻译、安全域和中断处理。该工件作为开源软件提供。

与 x86 Ring、ARM TrustZone 和 RISC-V PMP 相比, RPA 为虚拟化、机密计算和沙箱场景提供了统一抽象。与 CHERI 相比, RPA 解决特权层级而非内存安全; 两种机制互补。

未来工作包括 Sail 形式化规范、Isabelle 安全属性证明、gem5 性能估计模拟、FPGA 原型和操作系统集成。我们相信 RPA 为需要灵活、可扩展隔离的未来处理器架构提供了有前景的方向。

参考文献

- [1] RPA Project, “RPA Python Simulator: Executable Specification of Recursive Privilege Architecture,” <https://github.com/rpa-project/rpa-pysim>, 2024, version 1.0.
- [2] L. Soares and M. Stumm, “FlexSC: Flexible system call scheduling with exception-less system calls,” in *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010, pp. 1–8.
- [3] *ARM TrustZone Technology*, ARM Limited, 2023, aRM Developer Documentation.
- [4] *Intel 64 and IA-32 Architectures Software Developer’s Manual*, Intel Corporation, 2023, order Number: 325384-079US.
- [5] *Intel Trust Domain Extensions Architecture Specification*, Intel Corporation, 2023.
- [6] R. N. M. Watson, J. Woodruff, P. G. Neumann, S. W. Moore, J. Anderson, D. Chisnall, N. Dave, B. Davis, K. Gudka, B. Laurie *et al.*, “CHERI: A hybrid capability-system architecture for scalable software compartmentalization,” *IEEE Symposium on Security and Privacy*, pp. 20–37, 2015.
- [7] M. Ben-Yehuda, M. D. Day, Z. Dubitzky, M. Factor, N. Har’El, A. Gordon, A. Liguori, O. Wasserman, and B.-A. Yassour, “The turtles project: Design and implementation of nested virtualization,” in *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010, pp. 423–436.
- [8] CHERI Project, “CHERI Sail Model,” <https://github.com/CTSRD-CHERI/cheri-sail-ml>, 2023, accessed: 2024.
- [9] RISC-V International, “RISC-V Sail Model,” <https://github.com/riscv/sail-riscv>, 2023, accessed: 2024.
- [10] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish *et al.*, “seL4: Formal verification of an os kernel,” in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, 2009, pp. 207–220.
- [11] “Performance Best Practices for VMware vSphere 7.0,” Technical White Paper, 2022.
- [12] *The RISC-V Instruction Set Manual Volume II: Privileged Architecture Version 1.12*, RISC-V International, 2021, document Version 20211203.